CHAPTER 1

INTRODUCTION

Agriculture plays a vital role in the economic development of a country and serves as one of the primary sources of food, employment, and livelihood for millions of people. It contributes significantly to national income, supports rural communities, and provides essential raw materials for various industries. Agricultural products pass through several stages before reaching consumers, including cultivation, harvesting, storage, transportation, distribution, and retailing. The entire process is known as the agricultural food supply chain. It involves multiple stakeholders, such as farmers, distributors, retailers, administrators, and consumers, who work together to ensure that agricultural products are delivered efficiently from their source to their final destination. Effective management of this supply chain is essential for maintaining product quality, ensuring timely delivery, reducing information gaps, and improving coordination among the participants involved.

In the present digital era, the management of agricultural food supply chains has become increasingly important due to the growing demand for transparency, product quality, and reliable information. Consumers are becoming more aware of the origin, quality, safety, and handling of the food products they purchase. They expect accurate information about where agricultural products are cultivated, how they are transported, and through which distribution channels they reach the market. Similarly, farmers and other supply-chain participants require reliable systems to manage product information, monitor product movement, and maintain accurate transaction records. However, many traditional agricultural supply-chain systems still depend on manual documentation, paper-based records, and centralized information systems. These approaches can create difficulties in maintaining transparency, verifying product origins, tracking product movement, and sharing reliable information among different stakeholders.

One of the major challenges in conventional agricultural supply chains is the lack of a unified and transparent information management system. Information recorded by one participant may not be easily accessible or verifiable by another participant. This can result in fragmented records, delays in communication, difficulties in identifying the actual origin of products, and reduced accountability. For example, when an agricultural product moves from a farmer to a distributor and then to a retailer, information about each stage may be stored separately. If a problem occurs during transportation or distribution, identifying the responsible participant and verifying the product's movement history can become difficult. Furthermore, centralized information systems may be

vulnerable to unauthorized modifications, data manipulation, and technical failures. These limitations highlight the need for a secure, distributed, and reliable framework that can improve the management of agricultural food supply chains.

To address these challenges, the proposed project, Distributed Intelligence Framework for End-to-End Agricultural Food Supply Traceability, Transparency and Secure Governance, introduces a blockchain-based approach to agricultural supply-chain management. The framework integrates blockchain technology, smart contracts, QR code-based product identification, and web-based applications to establish a transparent and secure environment for recording and tracking agricultural products. Blockchain technology provides a distributed digital ledger in which important transactions and product movement records can be stored in a tamper-resistant manner. Unlike conventional centralized databases, blockchain maintains transaction records through a distributed network and uses cryptographic mechanisms to protect the integrity of recorded information. Once transactions are successfully validated and added to the blockchain, modifying previously recorded information becomes difficult. This feature helps maintain a consistent history of agricultural products and allows authorized participants to verify the recorded information.

The proposed framework focuses on improving the complete journey of agricultural products, starting from farmers and continuing through distributors and retailers until the products reach consumers. Initially, farmers can register themselves in the system and provide the required details for creating an account. After successful registration and authorization, farmers can register agricultural products by entering relevant information such as product name, quantity, origin, and other necessary details. Each registered product can be organized into a product batch with a unique identification. This helps distinguish one batch from another and provides a structured method of maintaining product-related information. The registration process establishes the initial record of an agricultural product and creates the foundation for tracking its movement throughout the supply chain.

After product registration, distributors play an important role in managing the movement of agricultural products from farms to distribution locations and retail outlets. They can record product pickup, transportation, and delivery-related updates through the application. These updates help maintain information about the movement of agricultural batches and provide visibility into the activities performed during distribution. When the products reach retailers, the retailers can record the receipt of agricultural batches and update their availability for consumers. Each stage contributes to the overall product history and helps establish a sequence of recorded supply-chain

activities. By maintaining these records in an organized manner, the framework aims to reduce information gaps and improve coordination among different participants.

A major feature of the proposed framework is the integration of smart contracts for managing important supply-chain operations. Smart contracts are programs deployed on a blockchain network that automatically execute predefined operations when specified conditions are satisfied. They help establish rules for processing transactions and managing interactions between different stakeholders. In the proposed system, smart contracts are developed using Solidity, a programming language commonly used for creating blockchain-based applications. These contracts define the operations associated with product registration, batch management, transaction recording, and supply-chain updates. They also support the implementation of access permissions according to the responsibilities of different users. By using smart contracts, the framework aims to maintain consistency in transaction processing and reduce the possibility of unauthorized changes to recorded supply-chain information.

The proposed system uses Polygon-compatible blockchain infrastructure for deploying and interacting with smart contracts. Polygon provides a blockchain environment that supports decentralized applications and smart contract execution. It is suitable for developing applications that require blockchain-based transaction processing and interaction between users and distributed networks. The integration of Polygon allows important supply-chain activities to be recorded as blockchain transactions. These transactions provide a verifiable history of the activities performed by different stakeholders. The use of blockchain infrastructure also supports the development of a decentralized approach to managing agricultural product information instead of depending entirely on a single centralized authority.

Web3 integration is another important component of the proposed framework. Web3 technology provides communication between the web application and the blockchain network, allowing users to interact with deployed smart contracts through a user-friendly interface. Web3.js can be used to connect the application with the blockchain network, retrieve smart contract information, and submit transactions for processing. Through this integration, users can perform their authorized activities without directly interacting with the underlying blockchain infrastructure. The web application acts as an interface between stakeholders and the blockchain network, making the system easier to access and operate. It combines the functionality of conventional web applications with the security and transparency features of blockchain technology.

QR code technology is incorporated into the proposed framework to improve product identification and consumer access to supply-chain information. A Quick Response code is a two-dimensional barcode that can store information or provide access to a specific digital resource when scanned using a suitable device. In the proposed system, each registered product batch can be associated with a unique QR code. This QR code acts as a connection between the physical agricultural product and its digital information maintained by the application. When consumers scan the QR code using a smartphone or another compatible device, they can access the available information associated with the product batch. This information may include product details, origin information, and the recorded supply-chain history. The QR code feature provides a convenient method of retrieving product information without requiring consumers to have direct knowledge of blockchain technology.

The integration of QR codes with blockchain-based product records improves the accessibility of traceability information and helps consumers make more informed purchasing decisions. It provides greater visibility into the movement of agricultural products and allows consumers to understand the stages through which a product has passed before reaching the retail market. It also helps authorized stakeholders identify product batches and retrieve their associated records when required. However, the reliability of traceability information depends on the accuracy of the details initially entered by authorized participants. Blockchain helps protect recorded information against unauthorized alteration, but it cannot independently guarantee that all physical-world information entered into the system is correct. Therefore, proper verification and responsible data entry remain important aspects of the proposed framework.

Secure governance is another essential aspect of the proposed project. Since agricultural supply chains involve multiple participants with different responsibilities, the system must provide appropriate access permissions to prevent unauthorized activities. The proposed framework incorporates role-based access control to manage the activities of farmers, distributors, retailers, and administrators. Each user is assigned a role according to their responsibilities within the supply chain, and access to specific functions is provided based on the assigned role. Farmers are permitted to register agricultural products and create product batches, while distributors can manage pickup, transportation, and delivery updates. Retailers can record product receipt and manage product availability. Administrators are responsible for managing user-related activities, approvals, and system-level controls. This structured access mechanism helps ensure that users perform only the activities permitted under their respective roles.

The system also incorporates user registration, authentication, and administrative approval mechanisms to improve access management. These features help identify users and control their participation in the agricultural supply-chain process. Administrative approval can be used to verify and authorize participants before they are allowed to perform specific operations. By maintaining structured records of product batches and transactions, the framework supports better accountability and provides a clear history of activities performed by different stakeholders. The combination of role-based permissions and blockchain-based transaction records aims to establish a more controlled and reliable environment for managing agricultural supply-chain information.

Another important objective of the proposed framework is to improve communication and information sharing among supply-chain participants. In traditional systems, stakeholders may depend on separate databases, physical documents, or direct communication to exchange product-related information. This can result in delays, inconsistencies, and difficulties in verifying information. The proposed application provides a common digital platform through which authorized users can access relevant product and transaction details. By connecting different participants through a structured system, it aims to improve coordination and reduce the difficulties associated with fragmented information management. The availability of product history also supports better monitoring and helps stakeholders understand the movement of agricultural products across different stages.

The proposed framework is designed to combine the advantages of blockchain technology with the accessibility of web-based applications. Blockchain provides a distributed and tamper-resistant method of recording important transactions, while smart contracts define the rules for performing supply-chain operations. Polygon-compatible infrastructure supports the deployment and execution of these contracts, and Web3 integration enables communication between the application and the blockchain network. QR codes provide a simple method of identifying product batches and retrieving available product information. Role-based access control ensures that different stakeholders can access the functions required for their responsibilities. Together, these components form an integrated framework for managing agricultural product information and improving traceability throughout the supply chain.

The significance of the proposed project lies in its ability to address important limitations of conventional agricultural supply-chain management. By providing end-to-end product tracking, the framework aims to improve visibility into the movement of agricultural products. Blockchain-based records support transaction integrity and make unauthorized modifications more difficult. Smart

contracts help manage predefined operations, while QR code technology improves access to product information for consumers. Role-based governance supports accountability and helps control the activities of different stakeholders. These features can contribute to improved trust, coordination, and transparency among farmers, distributors, retailers, administrators, and consumers.

The framework also provides opportunities for future improvements through the integration of additional digital technologies. Internet of Things devices can be incorporated to monitor environmental conditions such as temperature, humidity, and storage conditions during transportation and storage. This information could help stakeholders monitor the handling of agricultural products and identify conditions that may affect product quality. Artificial intelligence and data analytics could also be introduced to analyze supply-chain activities, identify patterns, and support better decision-making. Automated payment mechanisms and additional smart contract functionalities could further improve transaction management and operational efficiency. Such enhancements could extend the capabilities of the framework and support the development of a more intelligent and sustainable agricultural supply chain.

Overall, the Distributed Intelligence Framework for End-to-End Agricultural Food Supply Traceability, Transparency and Secure Governance aims to establish a transparent, secure, and accountable agricultural food supply-chain management environment. By integrating blockchain technology, smart contracts, Polygon infrastructure, Solidity programming, Web3 communication, QR code-based identification, and role-based governance, the system provides a structured approach to managing agricultural products from their origin to their final destination. It addresses the challenges associated with fragmented records, limited product visibility, and inadequate accountability in conventional supply-chain systems. The proposed framework provides a foundation for improving product traceability, strengthening information sharing, and increasing consumer confidence in agricultural products. Through further development and integration of emerging technologies, it has the potential to contribute to a more reliable, efficient, and sustainable agricultural food supply-chain ecosystem.

CHAPTER 2
LIERATURE SURVEY

2.1) Using Blockchain and Smart Contract for Traceability in Agricultural Products Supply Chain

Authors: Chuntang Yu, Yongzhao Zhan, and Zhiyuan Li (2020).

This paper examines the use of blockchain technology and smart contracts to improve traceability in agricultural product supply chains. It focuses on maintaining reliable records of agricultural products as they move between different participants. Blockchain can help preserve transaction histories and make product information easier to verify. Smart contracts can support the recording of important supply-chain events and transfers. The study is relevant to the proposed project because AgriChain uses blockchain-based records to track agricultural product batches from farmers through distributors and retailers, helping improve transparency, accountability, and trust.

2.2) Survey on Agri-Food Supply Chain Using Blockchain

Authors: Dayana D. S. and Kalpana G. (2021).

This paper reviews the application of blockchain technology in agricultural and food supply chains. It discusses how distributed records can improve traceability, protect stored transaction information from unauthorized alteration, and support information sharing among supply-chain participants. The study also considers the potential use of smart contracts to automate and manage supply-chain activities. These concepts are relevant to the proposed AgriChain project because it aims to provide a transparent record of agricultural product movements and allow authorized stakeholders and consumers to access product history.

2.3) The Role of Blockchain for Sustainable Agri-Food Supply Chain Management

Authors: S. S. Sambare and co-authors (2022).

This paper focuses on the role of blockchain technology in developing sustainable and accountable agricultural food supply chains. It explores how shared and verifiable records can improve coordination between participants, increase transparency, and support the tracking of products throughout their journey. Blockchain-based information can help stakeholders understand product movement and identify where supply-chain problems may occur. The findings relate to the proposed

project because AgriChain records product batch activities and stakeholder hand-offs, helping improve visibility, accountability, and the management of agricultural products.

2.4) Blockchain Technology in Agriculture Product Supply Chain

Authors: Shivendra and co-authors (2021).

This paper studies the application of blockchain technology to agricultural product supply chains. It discusses the use of distributed transaction records to support product tracking and improve the availability of supply-chain information. By maintaining a shared history of recorded activities, blockchain can help different participants verify product movement and reduce dependence on fragmented manual records. The study provides a foundation for the proposed AgriChain system, which allows farmers to create product batches, distributors and retailers to update product status, and consumers to view traceability information through QR codes.

2.5) An Overview of Blockchain Technology: Architecture, Consensus, and Future Trends

Authors: Zheng et al. (2017).

This paper provides an overview of blockchain technology, including its architecture, consensus mechanisms, and future development trends. It explains the fundamental concepts behind distributed ledgers and how transactions can be recorded and verified across a network. Understanding these concepts is important when developing applications that require reliable transaction histories and shared information. The study supports the technical foundation of the proposed AgriChain project, which uses a blockchain platform and smart contracts to record agricultural supply-chain events and provide a verifiable history of product movement.

2.6) Blockchain Technology for Secure Supply Chain Management: A Comprehensive Review

Authors: Agarwal et al. (2022).

This paper reviews the use of blockchain technology for secure supply-chain management. It examines how distributed records can support transparency, transaction auditing, and information sharing between organizations. Blockchain can help participants inspect recorded activities and establish a clearer history of transactions across different stages of a supply chain. These concepts are relevant to the proposed AgriChain project because it aims to provide secure and traceable agriculturalproductrecords.

CHAPTER 3
SYSTEM ANALYSIS AND REQUIREMENTS

The Distributed Intelligence Framework for End-to-End Agricultural Food Supply Traceability, Transparency and Secure Governance is developed to improve the tracking and management of agricultural products throughout the supply chain. The proposed AgriChain system uses a web-based application integrated with blockchain technology to maintain product batch records and provide traceability information to different stakeholders. The system supports farmers, distributors, retailers, consumers, and administrators through role-based access. Farmers can create agricultural product batches, distributors can manage shipment activities, and retailers can confirm product receipt and availability.

3.1 Existing System

Traditional agricultural supply chains mainly depend on paper-based records, manual documentation, and centralized information systems. Product information is maintained separately by farmers, distributors, retailers, and other participants. As a result, tracking agricultural products from their origin to the final consumer can become difficult and time-consuming.

Limitations of Existing System

- Limited visibility into agricultural product origin and movement.
- Dependence on manual records and centralized information systems.
- Possibility of inconsistent or altered records.
- Time-consuming product tracking and information verification.
- Difficulty coordinating farmers, distributors, and retailers.
- Increased administrative effort and potential disputes.

3.2 Proposed System

The proposed AgriChain system integrates a web-based application with blockchain technology to improve agricultural product traceability, transparency, and secure governance. It maintains records of important product batch activities and provides different functions according to each stakeholder's role.

Advantages of the Proposed System

- Provides batch-level traceability from farmers to consumers.
- Improves transparency across agricultural supply-chain activities.
- Maintains a verifiable history of recorded product movements.
- Uses role-based access to separate stakeholder responsibilities.
- Supports administrator approval and user management.
- Generates QR codes for convenient consumer access to product history.
- Reduces dependence on fragmented manual records.
- Improves coordination between farmers, distributors, and retailers.

3.3 System Specification:

HARDWARE REQUIREMENTS:

- **System :** Intel i3
- **Hard Disk :** 1 TB.
- **Monitor** : 14’ Colour Monitor.
- **Mouse :** Optical Mouse.
- **Ram :** 4GB.

SOFTWARE REQUIREMENTS:

- **Operating system :** Windows 10.
- **Coding Language :** Python.
- **Front-End :** Html. CSS
- **Designing :** Html,css,javascript.
- **Data Base :** SQLite.

CHAPTER 4
SYSTEM STUDY AND DESIGN

4.1 Feasibility Study

Technical Feasibility

The AgriChain system uses Python, Django, HTML, CSS, JavaScript, Web3.py/Web3.js, and SQLite. Blockchain integration with Polygon/Ethereum supports recording and retrieving agricultural product traceability information. These technologies are suitable for developing the proposed web-based system.

Economic Feasibility

The project uses open-source technologies and development tools, which help reduce software licensing costs. It can also reduce dependence on manual records and simplify product tracking. However, hosting and blockchain transaction fees may involve additional costs.

Operational Feasibility

The system provides separate functions for farmers, distributors, retailers, consumers, and administrators. Its user-friendly interface supports product batch creation, shipment tracking, inventory management, and QR-code-based traceability.

Schedule Feasibility

The project can be developed in stages, including web application development, database setup, blockchain integration, QR-code generation, and testing, using the available development tools

4.2 System Architecture

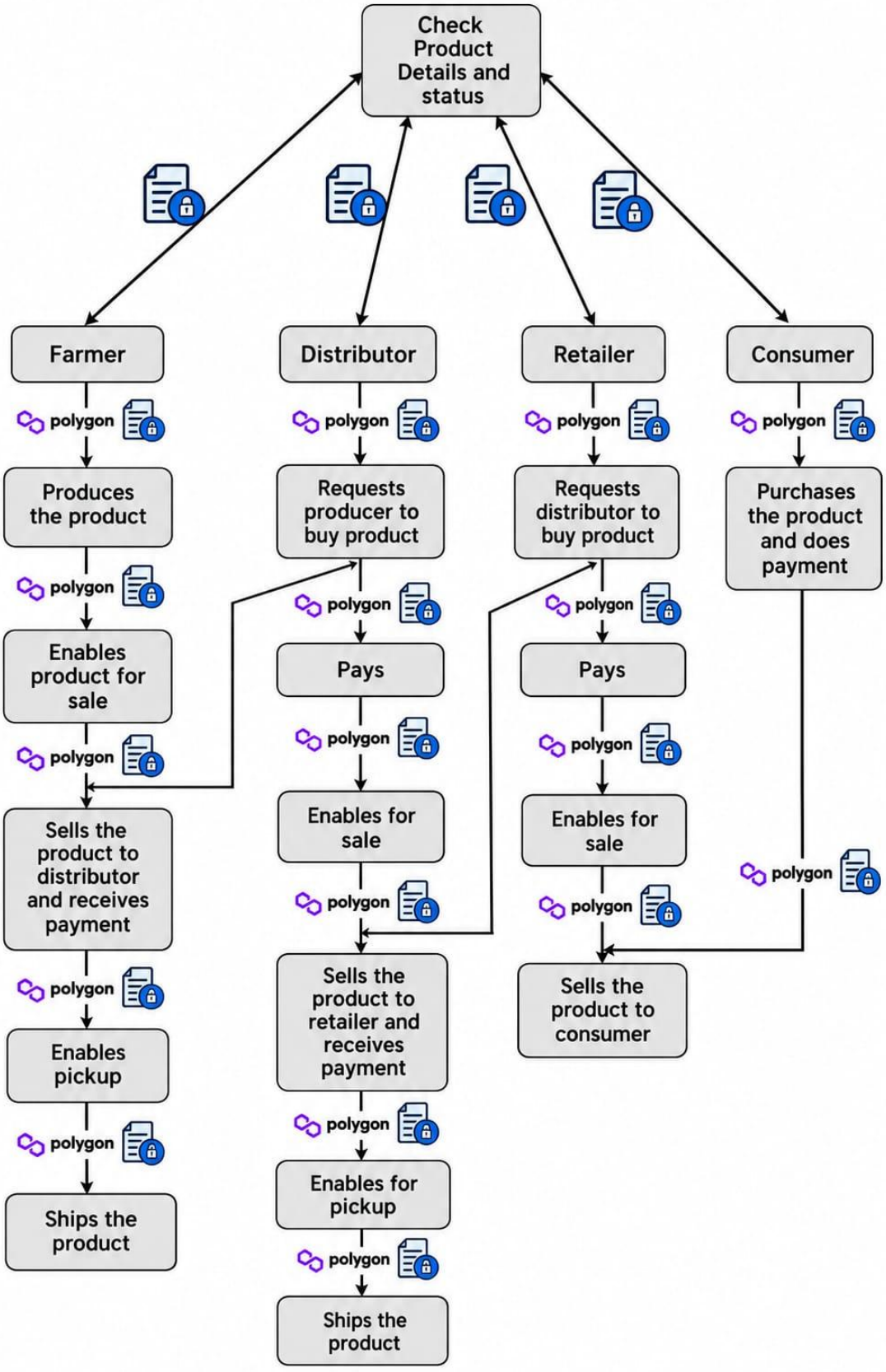


Figure 4.1 Agrichain System Architecture

4.2.1 Data Flow Diagram:

1. The DFD is also called as bubble chart. It is a simple graphical formalism that can be used to represent a system in terms of input data to the system, various processing carried out on this data, and the output data is generated by this system.

2. The data flow diagram (DFD) is one of the most important modeling tools. It is used to model the system components. These components are the system process, the data used by the process, an external entity that interacts with the system and the information flows in the system.

3. DFD shows how the information moves through the system and how it is modified by a series of transformations. It is a graphical technique that depicts information flow and the transformations that are applied as data moves from input to output.

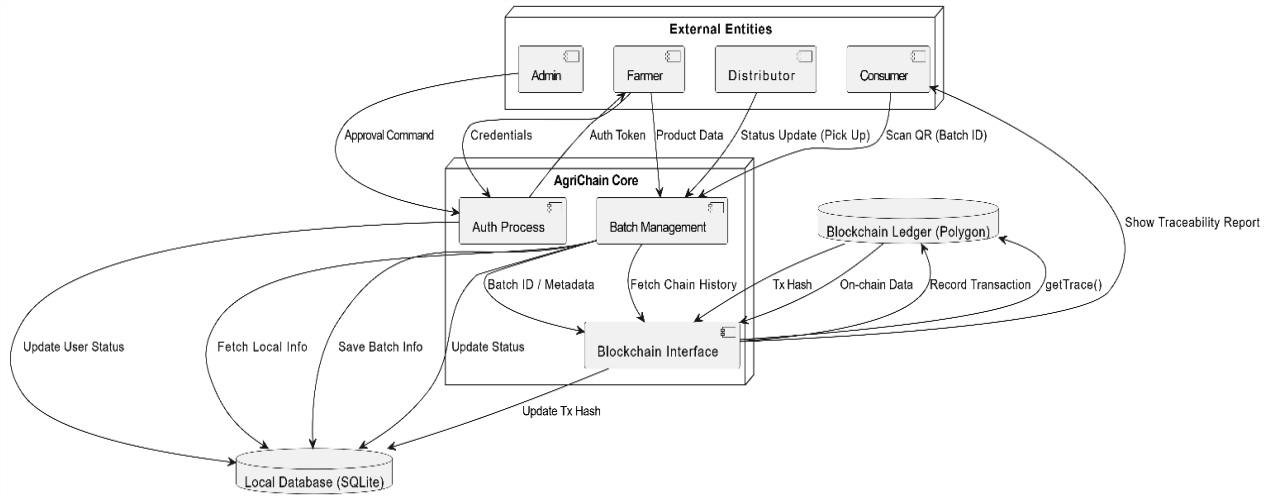


Figure 4.2 Data Flow Diagram

4.2.2 Use Case Diagram:

A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.

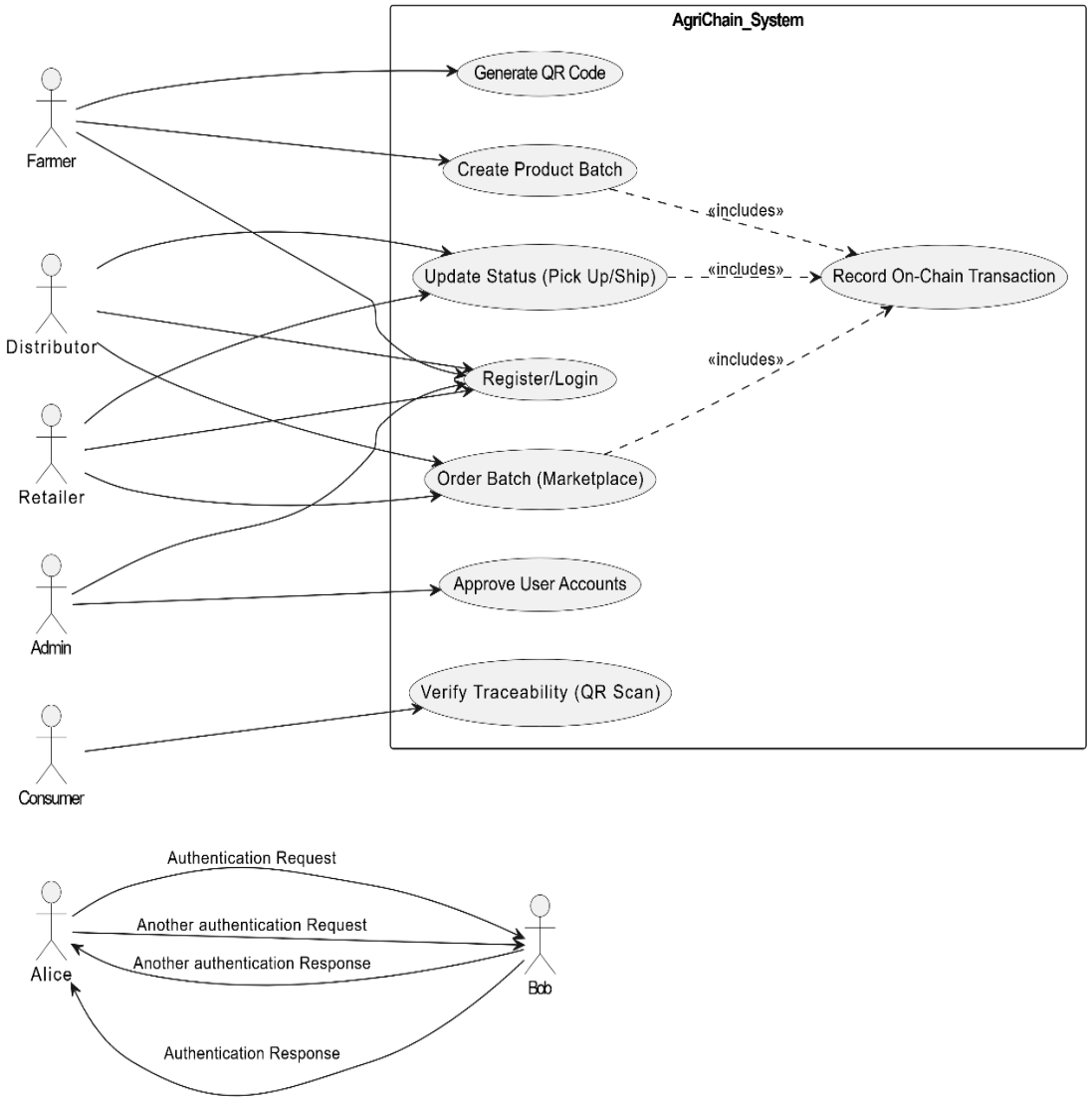


Figure 4.3 Use Case Diagram

4.2.3 Class Diagram:

In software engineering, a class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information.

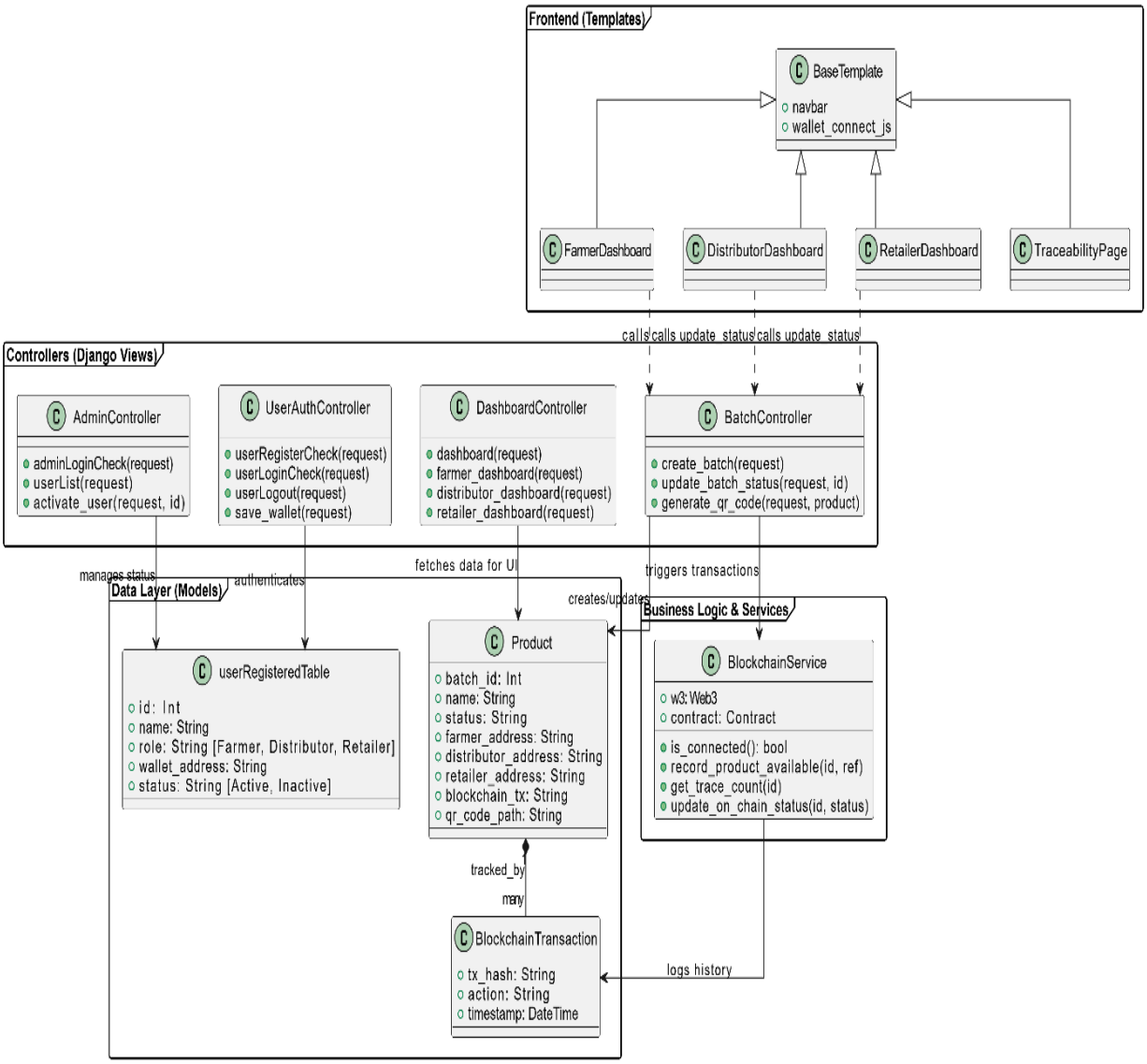


Figure 4.4 Class Diagram

4.2.4 Sequence Diagram:

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams.

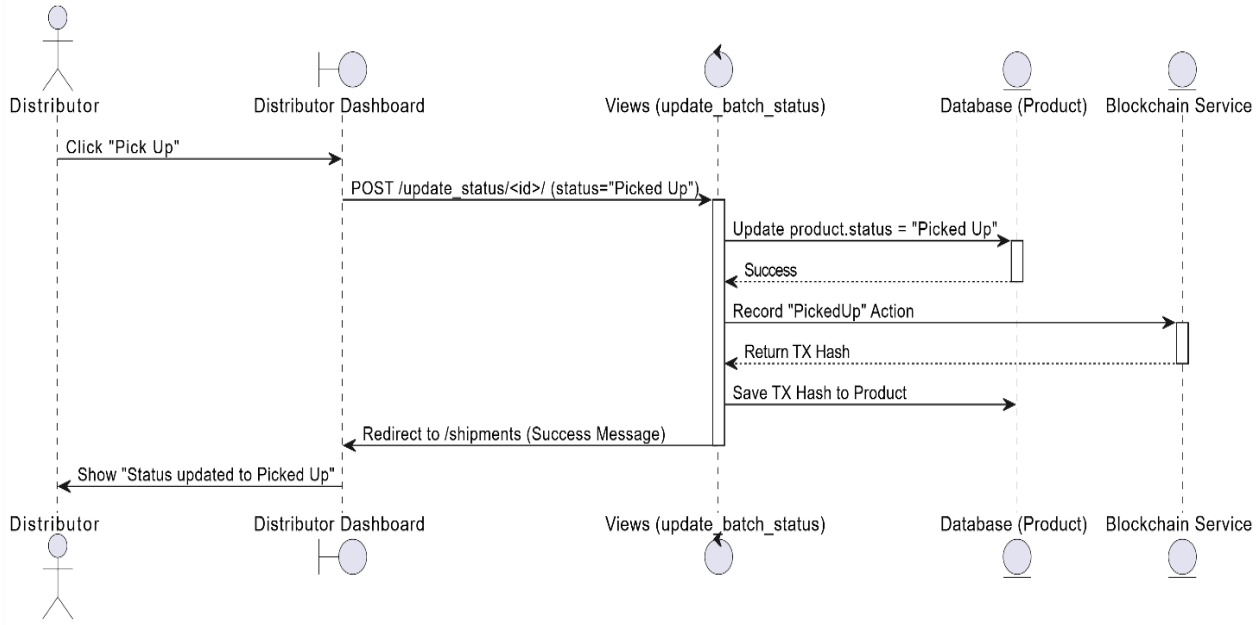


Figure 4.5 Sequence Diagram

CHAPTER 5

INPUT AND OUTPUT DESIGN

5.1 Input Design

Input design is one of the most important parts of any software system. It focuses on collecting accurate data from users and providing a simple and user-friendly interface for interaction. In this project, the input design is developed to allow farmers, distributors, retailers, consumers, and administrators to interact with the AgriChain system for agricultural product tracking, supply-chain management, and traceability.

The system accepts the following inputs:

1. User Registration Input

New users must enter the following details during registration:

- Username
- Email ID
- Password
- Confirm Password
- User Role

The system validates the entered details to ensure correct registration. New accounts may require administrator activation before access is granted.

2. User Login Input

Registered users can log into the system using:

- Email ID / Login ID
- Password

Authentication is performed to provide secure access to the application and the appropriate role-based dashboard.

3. Farmer Product Batch Input

Farmers can enter agricultural product details, including:

- Product / Crop Name
- Batch Details
- Quantity
- Harvest Date

The entered information is used to create a product batch and make it available for distribution.

4. Distributor and Retailer Input

Distributors can select available batches and update shipment status, such as pickup, shipped, and received. Retailers can confirm product receipt and update product availability for consumers.

5. Administrator Input

The admin module can manage:

- User Details
- User Activation and Deactivation
- Stakeholder Access
- Platform Activities

The administrator oversees user access and manages the participants registered on the platform.

5.2 Output Design

Output design refers to the way the system presents processed information to users. The output should be accurate, understandable, and visually clear. In this project, the output design provides product batch details, shipment status, inventory information, and traceability records through a web-based interface.

1. User Dashboard Output

After successful login, the system displays the appropriate dashboard according to the user's role, providing access to permitted functions

2. Product Batch Output

The farmer dashboard displays created product batches, their details, quantity, and availability status.

3. Shipment Status Output

The distributor dashboard displays shipment information and the current status of product batches, including pickup, shipped, and received.

4. Inventory Output

The retailer dashboard displays received products and their availability status for consumers.

5. QR Code and Traceability Output

The system generates a QR code associated with a product batch. Consumers can scan or open the QR code to view product details and the recorded supply-chain history.

6. Admin Output Reports

The admin module displays information such as:

- Registered User Details
- Stakeholder Information
- User Activation Status
- User Management Controls

The overall input and output design of the AgriChain system provides a user-friendly environment for managing agricultural product batches, tracking supply-chain activities, and improving transparency through blockchain-based traceability.

CHAPTER 6

SOFTWARE ENVIRONMENT

PYTHON

Python is a general-purpose interpreted, interactive, object-oriented, and high-level programming language. An interpreted language, Python has a design philosophy that emphasizes code readability (notably using whitespace indentation to delimit code blocks rather than curly brackets or keywords), and a syntax that allows programmers to express concepts in fewer lines of code than might be used in languages such as C++or Java. It provides constructs that enable clear programming on both small and large scales. Python interpreters are available for many operating systems. CPython, the reference implementation of Python, is open source software and has a community-based development model, as do nearly all of its variant implementations. CPython is managed by the non-profit Python Software Foundation.

Interactive Mode Programming

Invoking the interpreter without passing a script file as a parameter brings up the following prompt –

```
$ python
Python 2.4.3 (#1, Nov 11 2010, 13:34:43)
[GCC 4.1.2 20080704 (Red Hat 4.1.2-48)] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

Type the following text at the Python prompt and press the Enter −

```
>>> print "Hello, Python!"
```

Script Mode Programming

Invoking the interpreter with a script parameter begins execution of the script and continues until the script is finished. When the script is finished, the interpreter is no longer active.

Let us write a simple Python program in a script. Python files have extension .py. Type the following source code in a test.py file −

Live Demo

```
print "Hello, Python!"
```

We assume that you have Python interpreter set in PATH variable. Now, try to run this program as follows −

$ python test.py

This produces the following result −

Hello, Python!

Let us try another way to execute a Python script. Here is the modified test.py file −

Live Demo

```
#!/usr/bin/python
print "Hello, Python!"
```

We assume that you have Python interpreter available in /usr/bin directory. Now, try to run this program as follows −

```
$ chmod +x test.py     # This is to make file executable
$./test.py
```

This produces the following result −

Hello, Python!

Python Identifiers

A Python identifier is a name used to identify a variable, function, class, module or other object. An identifier starts with a letter A to Z or a to z or an underscore (_) followed by zero or more letters, underscores and digits (0 to 9).

Python does not allow punctuation characters such as @, $, and % within identifiers. Python is a case sensitive programming language. Thus, Manpower and manpower are two different identifiers in Python.

Reserved Words

The following list shows the Python keywords. These are reserved words and you cannot use them as constant or variable or any other identifier names. All the Python keywords contain lowercase letters only.

```
and       exec    not
assert    finally or
break     for     pass
class     from    print
continue          global  raise
def       if      return
del       import  try
elif      in      while
else      is      with
except    lambdayield
```

Lines and Indentation

Python provides no braces to indicate blocks of code for class and function definitions or flow control. Blocks of code are denoted by line indentation, which is rigidly enforced.

The number of spaces in the indentation is variable, but all statements within the block must be indented the same amount. For example −

```
if True:
   print "True"
else:
   print "False"
```

However, the following block generates an error −

```
if True:
print "Answer"
print "True"
else:
print "Answer"
print "False"
```

Thus, in Python all the continuous lines indented with same number of spaces would form a block.

Quotation in Python

Python accepts single ('), double (") and triple ('" or """) quotes to denote string literals, as long as the same type of quote starts and ends the string.

The triple quotes are used to span the string across multiple lines. For example, all the following are legal −

```
word = 'word'
sentence = "This is a sentence."
paragraph = """This is a paragraph. It is
made up of multiple lines and sentences."""
```

Comments in Python

A hash sign (#) that is not inside a string literal begins a comment. All characters after the # and up to the end of the physical line are part of the comment and the Python interpreter ignores them.

Live Demo

```
#!/usr/bin/python
# First comment
```

print "Hello, Python!" # second comment

This produces the following result −

Hello, Python!

You can type a comment on the same line after a statement or expression −

name = "Madisetti" # This is again comment

You can comment multiple lines as follows −

This is a comment.

This is a comment, too.

This is a comment, too.

I said that already.

Following triple-quoted string is also ignored by Python interpreter and can be used as a multiline comments:

'''

This is a multiline

comment.

'''

Using Blank Lines

A line containing only whitespace, possibly with a comment, is known as a blank line and Python totally ignores it.

In an interactive interpreter session, you must enter an empty physical line to terminate a multiline statement.

Waiting for the User

The following line of the program displays the prompt, the statement saying “Press the enter key to exit”, and waits for the user to take action −

#!/usr/bin/python

raw_input("\n\nPress the enter key to exit.")

Here, "\n\n" is used to create two new lines before displaying the actual line. Once the user presses the key, the program ends. This is a nice trick to keep a console window open until the user is done with an application.

Multiple Statements on a Single Line

The semicolon (;) allows multiple statements on the single line given that neither statement starts a new code block. Here is a sample snip using the semicolon.

import sys; x = 'foo'; sys.stdout.write(x + '\n')

Multiple Statement Groups as Suites

A group of individual statements, which make a single code block are called suites in Python. Compound or complex statements, such as if, while, def, and class require a header line and a suite. Header lines begin the statement (with the keyword) and terminate with a colon (:) and are followed by one or more lines which make up the suite. For example −

```
if expression :
   suite
elif expression :
   suite
else :
   suite
```

Command Line Arguments

Many programs can be run to provide you with some basic information about how they should be run. Python enables you to do this with -h −

```
$ python -h
usage: python [option] ... [-c cmd | -m mod | file | -] [arg] ...
Options and arguments (and corresponding environment variables):
-c cmd : program passed in as string (terminates option list)
-d     : debug output from parser (also PYTHONDEBUG=x)
-E     : ignore environment variables (such as PYTHONPATH)
-h     : print this help message and exit
```

Python Lists

The list is a most versatile datatype available in Python which can be written as a list of comma-separated values (items) between square brackets. Important thing about a list is that items in a list need not be of the same type.

Creating a list is as simple as putting different comma-separated values between square brackets. For example −

```
list1 = ['physics', 'chemistry', 1997, 2000];
list2 = [1, 2, 3, 4, 5 ];
list3 = ["a", "b", "c", "d"]
```

Creating a tuple is as simple as putting different comma-separated values. Optionally you can put these comma-separated values between parentheses also. For example −

```
tup1 = ('physics', 'chemistry', 1997, 2000);
tup2 = (1, 2, 3, 4, 5 );
```

tup3 = "a", "b", "c", "d";

The empty tuple is written as two parentheses containing nothing −

tup1 = ();

To write a tuple containing a single value you have to include a comma, even though there is only one value −

tup1 = (50,);

Like string indices, tuple indices start at 0, and they can be sliced, concatenated, and so on.

Accessing Values in Tuples

To access values in tuple, use the square brackets for slicing along with the index or indices to obtain value available at that index. For example −

Live Demo

```
#!/usr/bin/python
tup1 = ('physics', 'chemistry', 1997, 2000);
tup2 = (1, 2, 3, 4, 5, 6, 7 );
print "tup1[0]: ", tup1[0];
print "tup2[1:5]: ", tup2[1:5];
```

When the above code is executed, it produces the following result −

```
tup1[0]:  physics
tup2[1:5]:  [2, 3, 4, 5]
```

Updating Tuples

Accessing Values in Dictionary

To access dictionary elements, you can use the familiar square brackets along with the key to obtain its value. Following is a simple example −

Live Demo

```
#!/usr/bin/python
dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}
print "dict['Name']: ", dict['Name']
print "dict['Age']: ", dict['Age']
```

When the above code is executed, it produces the following result −

```
dict['Name']:  Zara
dict['Age']:  7
```

Updating Dictionary

You can update a dictionary by adding a new entry or a key-value pair, modifying an existing entry, or deleting an existing entry as shown below in the simple example −

Live Demo

```
#!/usr/bin/python
dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}
dict['Age'] = 8; # update existing entry
dict['School'] = "DPS School"; # Add new entry
print "dict['Age']: ", dict['Age']
print "dict['School']: ", dict['School']
```

When the above code is executed, it produces the following result −

```
dict['Age']:  8
dict['School']:  DPS School
```

Delete Dictionary Elements

You can either remove individual dictionary elements or clear the entire contents of a dictionary. You can also delete entire dictionary in a single operation.

To explicitly remove an entire dictionary, just use the del statement. Following is a simple example –

Live Demo

```
#!/usr/bin/python
dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}
del dict['Name']; # remove entry with key 'Name'
dict.clear();     # remove all entries in dict
del dict ;        # delete entire dictionary
print "dict['Age']: ", dict['Age']
print "dict['School']: ", dict['School']
```

This produces the following result. Note that an exception is raised because after del dict dictionary does not exist any more −

```
dict['Age']:
Traceback (most recent call last):
  File "test.py", line 8, in <module>
    print "dict['Age']: ", dict['Age'];
TypeError: 'type' object is unsubscriptable
```

Note − del() method is discussed in subsequent section.

Properties of Dictionary Keys

Dictionary values have no restrictions. They can be any arbitrary Python object, either standard objects or user-defined objects. However, same is not true for the keys.

There are two important points to remember about dictionary keys −

(a) More than one entry per key not allowed. Which means no duplicate key is allowed. When duplicate keys encountered during assignment, the last assignment wins. For example −

Live Demo

```
#!/usr/bin/python
dict = {'Name': 'Zara', 'Age': 7, 'Name': 'Manni'}
print "dict['Name']: ", dict['Name']
```

When the above code is executed, it produces the following result −

```
dict['Name']:  Manni
```

(b) Keys must be immutable. Which means you can use strings, numbers or tuples as dictionary keys but something like ['key'] is not allowed. Following is a simple example −

Live Demo

```
#!/usr/bin/python
dict = {['Name']: 'Zara', 'Age': 7}
print "dict['Name']: ", dict['Name']
```

When the above code is executed, it produces the following result −

```
Traceback (most recent call last):
  File "test.py", line 3, in <module>
    dict = {['Name']: 'Zara', 'Age': 7};
TypeError: unhashable type: 'list'
```

Tuples are immutable which means you cannot update or change the values of tuple elements. You are able to take portions of existing tuples to create new tuples as the following example demonstrates −

Live Demo

```
#!/usr/bin/python
tup1 = (12, 34.56);
tup2 = ('abc', 'xyz');
# Following action is not valid for tuples
# tup1[0] = 100;
# So let's create a new tuple as follows
tup3 = tup1 + tup2;
print tup3;
```

When the above code is executed, it produces the following result −

(12, 34.56, 'abc', 'xyz')

Delete Tuple Elements

Removing individual tuple elements is not possible. There is, of course, nothing wrong with putting together another tuple with the undesired elements discarded.

To explicitly remove an entire tuple, just use the del statement. For example −

Live Demo

```
#!/usr/bin/python
tup = ('physics', 'chemistry', 1997, 2000);
print tup;
```

DJANGO

Django is a high-level Python Web framework that encourages rapid development and clean, pragmatic design. Built by experienced developers, it takes care of much of the hassle of Web development, so you can focus on writing your app without needing to reinvent the wheel. It’s free and open source.

Django's primary goal is to ease the creation of complex, database-driven websites. Django emphasizes reusabilityand "pluggability" of components, rapid development, and the principle of don't repeat yourself. Python is used throughout, even for settings files and data models.

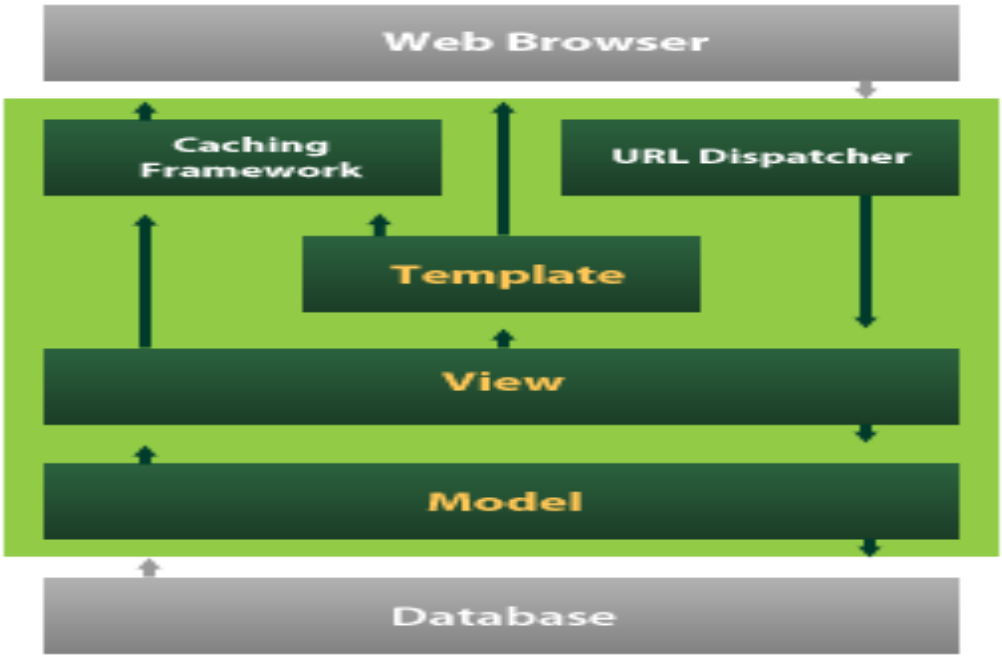


Figure 6.1 Django Flowchart

Django also provides an optional administrative create, read, update and delete interface that is generated dynamically through introspection and configured via admin models

Create a Project

Whether you are on Windows or Linux, just get a terminal or a cmd prompt and navigate to the place you want your project to be created, then use this code −

$ django-admin startproject myproject

This will create a "myproject" folder with the following structure −

```
myproject/
  manage.py
  myproject/
    __init__.py
    settings.py
    urls.py
    wsgi.py
```

The Project Structure

The “myproject” folder is just your project container, it actually contains two elements −

manage.py − This file is kind of your project local django-admin for interacting with your project via command line (start the development server, sync db...). To get a full list of command accessible via manage.py you can use the code −

$ python manage.py help

The “myproject” subfolder − This folder is the actual python package of your project. It contains four files −

__init__.py − Just for python, treat this folder as package.

settings.py − As the name indicates, your project settings.

urls.py − All links of your project and the function to call. A kind of ToC of your project.

wsgi.py − If you need to deploy your project over WSGI.

Setting Up Your Project

Your project is set up in the subfolder myproject/settings.py. Following are some important options you might need to set −

DEBUG = True

This option lets you set if your project is in debug mode or not. Debug mode lets you get more information about your project's error. Never set it to ‘True’ for a live project. However, this has to be set to ‘True’ if you want the Django light server to serve static files. Do it only in the development mode.

```
DATABASES = {
  'default': {
    'ENGINE': 'django.db.backends.sqlite3',
    'NAME': 'database.sql',
    'USER': '',
    'PASSWORD': '',
    'HOST': '',
    'PORT': '',
  }
}
```

CHAPTER 7

SYSTEM TESTING AND RESULTS DISCUSSION

7.1SYSTEM TESTING

Table 7.1 : System testing

Test Case ID	Test Description	Input	Expected Output	Actual Output	Status
TC01	User Registration	Valid user details	User registered successfully	User registered successfully	Pass
TC02	User Login	Valid login ID and password	User login successful	User login successful	Pass
TC03	User Login	Invalid login ID or password	Error message displayed	Error message displayed	Pass
TC04	Admin Login	Valid admin credentials	Admin login successful	Admin login successful	Pass
TC05	Admin Approval	Select pending user	User account activated	User account activated	Pass
TC06	Product Batch Creation	Product details	Product batch created successfully	Product batch created successfully	Pass
TC07	Blockchain Recording	Valid product transaction	Transaction recorded successfully	Transaction recorded successfully	Pass
TC08	QR Code Generation	Product batch details	QR code generated successfully	QR code generated successfully	Pass
TC09	Shipment Tracking	Shipment details	Shipment status displayed	Shipment status displayed	Pass
TC10	Product Traceability	Product batch ID	Product history displayed	Product history displayed	Pass
TC11	Retailer Confirmation	Received product details	Product receipt confirmed	Product receipt confirmed	Pass
TC12	Inventory Update	Product availability details	Inventory updated successfully	Inventory updated successfully	Pass
TC13	Product History	Existing batch records	Complete product history displayed	Complete product history displayed	Pass
TC14	Logout	User/Admin logout	Session terminated successfully	Session terminated successfully	Pass

7.1.2 SYSTEM TESTING TABLE

Table 7.2 System testing table

Test Case ID	Module Name	Test Description	Input	Expected Output	Actual Output	Status
TC01	Login Module	Check user login with valid credentials	Valid username and password	Login successful	Login successful	Pass
TC02	Login Module	Check login with invalid credentials	Invalid username or password	Error message displayed	Error message displayed	Pass
TC03	Registration Module	Register a new user	Valid registration details	User registered successfully	User registered successfully	Pass
TC04	Product Batch Module	Create a product batch	Crop name and quantity	Product batch created successfully	Product batch created successfully	Pass
TC05	Blockchain Module	Record a product transaction	Valid batch transaction	Transaction recorded successfully	Transaction recorded successfully	Pass
TC06	QR Code Module	Generate a product QR code	Product batch ID	QR code generated successfully	QR code generated successfully	Pass
TC07	Traceability Module	View product history	Product QR code	Product history displayed	Product history displayed	Pass
TC08	Shipment Module	Update shipment status	Pickup or delivery status	Shipment status updated successfully	Shipment status updated successfully	Pass
TC09	Retailer Module	Confirm product receipt	Received batch details	Product receipt confirmed	Product receipt confirmed	Pass
TC10	Admin Module	Approve a user account	Pending user account	User account activated	User account activated	Pass

7.2 RESULTS DISCUSSION

The proposed AgriChain system provides a blockchain-based solution for agricultural food supply chain traceability and transparency. It enables farmers to create product batches, distributors to update shipment details, and retailers to confirm product receipt. Consumers can access product information and trace its journey through QR codes, helping improve transparency and trust among all stakeholders.

The system uses blockchain technology to maintain transparent transaction records and supports the administrator in managing user accounts and access. By connecting farmers, distributors, retailers, and consumers through a common platform, AgriChain aims to improve traceability, accountability, and information sharing throughout the agricultural supply chain. The system helps consumers understand the journey of agricultural products from the farm to the final destination.

CHAPTER 8

CONCLUSION

AgriChain presents a blockchain-based solution for improving traceability, transparency, and accountability in the agricultural food supply chain. The system connects farmers, distributors, retailers, and consumers through a common platform. Farmers can create product batches, distributors can update shipment details, retailers can confirm product receipt, and consumers can access product information through QR codes. Blockchain and smart contracts are intended to maintain transparent transaction records and improve the reliability of product history.

The proposed system aims to reduce information gaps, improve data visibility, and build trust among supply chain participants. It also provides administrator functions for managing user accounts and access. By making agricultural product information easier to trace from the farm to the consumer, AgriChain can support better supply chain management and informed consumer decisions. Further improvements and comprehensive testing can help strengthen the system's security, reliability, and practical usability.

In the future, AgriChain can be enhanced by integrating IoT sensors for real-time monitoring of storage and transportation conditions, AI-based analytics for demand forecasting, and secure digital identity verification for supply chain participants. Additional improvements in smart-contract security, transaction efficiency, and user accessibility can make the system more reliable and scalable. These enhancements can further support the development of a transparent, secure, and efficient agricultural food supply chain.

CHAPTER 9
FURTHER ENHANCEMENT

The proposed AgriChain system can be further enhanced by integrating Internet of Things (IoT) sensors for real-time monitoring of agricultural products during transportation and storage. Temperature, humidity, and location data can be used to monitor product conditions and generate alerts when predefined limits are exceeded. Smart contracts can also be improved to automate payments after successful confirmation of product delivery.

The system can be integrated with decentralized identity and digital certification technologies to verify farmers, distributors, and retailers. Digital certificates for organic and fair-trade products can help improve product authenticity and consumer trust. Artificial Intelligence and Machine Learning techniques can also be introduced to forecast crop demand and improve supply chain planning.

Cloud-based deployment and mobile application support can be implemented to improve accessibility and enable the system to handle a larger number of users and transactions. Offline functionality can also be developed to allow stakeholders in areas with limited internet connectivity to record updates and synchronize them when the connection becomes available.

Future versions of AgriChain may include sustainability tracking, consumer feedback and reputation systems, automated dispute resolution, and integration with multiple blockchain networks. Augmented Reality can also be explored to provide consumers with interactive information about farms and product journeys through QR codes. These enhancements can improve the scalability, usability, transparency, and efficiency of agricultural food supply chain management.

REFERENCES

[1] C. Yu, Y. Zhan, and Z. Li, “Using Blockchain and Smart Contract for Traceability in Agricultural Products Supply Chain,” 2020 International Conference on Internet of Things and Intelligent Applications (ITIA), 2020.

[2] D. D. S. and K. G., “Survey on Agri-Food Supply Chain Using Blockchain,” 2021 Fifth International Conference on I-SMAC (IoT in Social, Mobile, Analytics and Cloud), 2021.

[3] S. S. Sambare, M. R. Borphalkar, A. V. Utekar, and V. Salunke, “The Role of Blockchain for Sustainable Agri-Food Supply Chain Management,” 2022 6th International Conference on Computing, Communication, Control and Automation (ICCUBEA), 2022.

[4] Shivendra, K. Chiranjeevi, M. K. Tripathi, and D. D. Maktedar, “Blockchain Technology in Agriculture Product Supply Chain,” 2021 International Conference on Artificial Intelligence and Smart Systems (ICAIS), 2021.

[5] Z. Zheng, S. Xie, H. Dai, X. Chen, and H. Wang, “An Overview of Blockchain Technology: Architecture, Consensus, and Future Trends,” 2017.

[6] World Economic Forum, “Realizing the Potential of Blockchain: A Multistakeholder Approach to the Stewardship of Blockchain and Cryptocurrencies,” 2018.

[7] U. Agarwal, V. Rishiwal, S. Tanwar, R. Chaudhary, G. Sharma, P. N. Bokoro, and R. Sharma, “Blockchain Technology for Secure Supply Chain Management: A Comprehensive Review,” 2022.

[8] P. Kumari, R. Kumar, A. Lohan, N. K. Patel, and P. Jain, “Designing Supply Chain Management System Using Blockchain: A Review,” World Conference on Communication & Computing (WCONF), Raipur, India, 2023.

[9] P. Dudczyk, J. K. Dunston, and G. V. Crosby, “Blockchain Technology for Global Supply Chain Management: A Survey of Applications, Challenges, Opportunities and Implications,” 2024.

[10] K. Gupta, L. Goel, M. Gupta, and N. Bansal, “Enhancing Supply Chain Management Using Blockchain Technology,” 2022 2nd International Conference on Intelligent Technologies (CONIT), Hubli, India, 2022.

[11] N. Patelli and M. Mandrioli, “Blockchain Technology and Traceability in the Agrifood Industry,” Journal of Food Science, vol. 85, no. 11, pp. 3670–3678, 2020. doi: 10.1111/1750-3841.15477.

[12] T. Bosona and G. Gebresenbet, “The Role of Blockchain Technology in Promoting Traceability Systems in Agri-Food Production and Supply Chains,” Sensors, vol. 23, no. 11, article 5342, 2023. doi: 10.3390/s23115342.

[13] R. M. Ellahi, L. C. Wood, and A. E. A. Bekhit, “Blockchain-Based Frameworks for Food Traceability: A Systematic Review,” Foods, vol. 12, no. 16, article 3026, 2023. doi: 10.3390/foods12163026.

[14] “Block Chain Based Agricultural Supply Chain—A Review,” Global Transitions Proceedings, vol. 2, no. 2, pp. 220–226, 2021. doi: 10.1016/j.gltp.2021.08.041.

[15] X. Peng, Z. Zhao, X. Wang, H. Li, J. Xu, and X. Zhang, “A Review on Blockchain Smart Contracts in the Agri-Food Industry: Current State, Application Challenges and Future Trends,” Computers and Electronics in Agricsulture, 2023. doi: 10.1016/j.compag.2023.107776.